Jetson 실행·ONNX/TensorRT 변환 절차서

엔진 모형 조립 검사 프로젝트 | 평가 참고 산출물 4·5 | v1.0

핵심 구현 성과 학습 모델의 PT → ONNX → TensorRT → Jetson 운영 package 를 해시로 연결하고, 현재 Jetson 에서 카메라·TensorRT 추론·3 프레임 판정·DB·PLC·웹을 하나의 서비스로 운용했다.

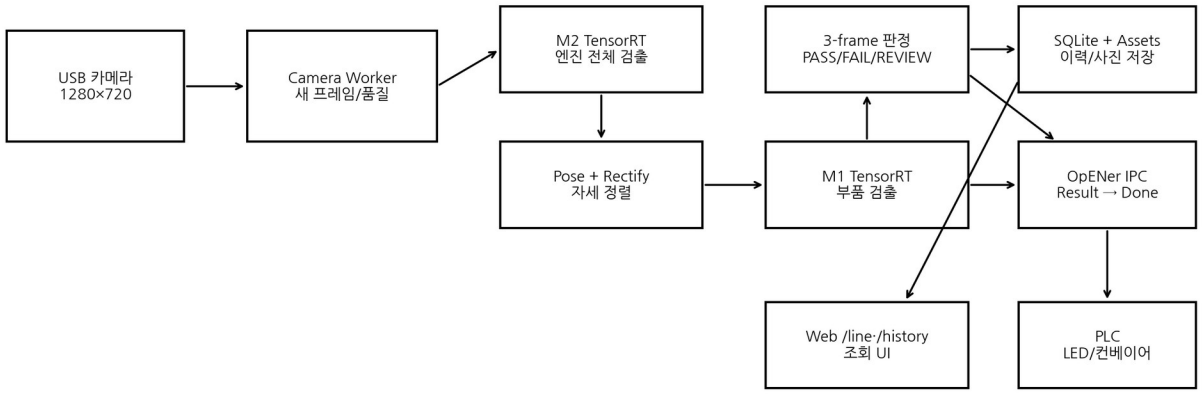
검토 항목	확인 결과
운영 기준	GitHub master cfaa372eece0cd6a7dc659cc316c256fb4b6af76 / 장치 release engine-integrated-32c484ac6aeb07f1
Jetson 환경	Ubuntu 22.04 · L4T 36.4.7 · Python 3.10.12 · CUDA 12.6 · TensorRT 10.3
현재 검출 모델	M1 dynamic_parts_005 + M2 envelope_v007_fp16 (TensorRT plan)
변환 계보	M1·M2 모두 PT → ONNX → TensorRT 의 SHA-256 연결 확인
실행 정책	detector_factory 에서 backend 를 명시 선택하며 silent fallback 을 금지
후속 검증	같은 Jetson Benchmark 에서 TensorRT 프로세스 RSS 관측 최대 약 49% 감소, E03 검출 성능은 PT 와 근접

본 문서는 “코드가 있다”는 사실보다, 어떤 코드가 어떤 계약과 모델을 읽고 실제 장치에서 어떻게 연결되는지 평가자가 추적할 수 있도록 구성했다.

01 현재 Jetson 운영 구조

운영 경로는 /home/jetson/oned_device_bench/current 이며 systemd 의 engine-inspector.service 하나가 전체 런타임을 감독한다. 장치별 설정과 모델은 config/·assets/, 검사 이력은 data/에 분리된다.

현재 Jetson 운영 흐름 (요약)



standalone_service.py가 단일 감독자로 카메라·추론·PLC 연계·웹을 관리

단계	주요 코드	역할
서비스 진입	jetson/standalone_service.py	설정 검증, 단일 소유권 lock, 카메라/추론/PLC/웹 런타임 구성
검출기 선택	jetson/src/vision/detector_factory.py	manifest 의 backend 를 명시적으로 선택. TensorRT 실패를 PyTorch 로 자동 대체하지 않음
엔진 전체·자세	jetson/src/vision/engine_inspector.py	M2 엔진 전체 검출 → pose 추정 → canonical rectification
부품 검출	동일 engine_inspector + M1 detector	정렬된 입력에서 필수 부품 검출 후 원본 좌표로 복원
판정	src/decision/engine_dynamic.py	3 프레임을 집계해 자리별 상태와 최종 판정 생성
PLC 연계	src/control/opener_cyclic.py / OpENer child	Result 이후 Done 순서로 결과 전달
저장·조회	journal/history + edge_service	SQLite·원본/표시 이미지 저장, /line·/history 조회

02 Jetson 실행 코드 맵

평가표의 “Jetson 실행 코드”는 하나의 스크립트가 아니라 카메라·검출·판정·저장·PLC·웹의 실행 경로 전체다. 아래 파일을 중심으로 읽으면 현재 운영 흐름을 재구성할 수 있다.

경로	평가 시 확인할 내용
jetson/standalone_service.py	운영 진입점. config·line profile·PLC 계약을 검증하고 runtime 및 웹 앱을 시작
jetson/src/vision/detector_factory.py	manifest backend 검증 후 TensorRT 또는 PyTorch adapter 생성. 암묵 fallback 없음
jetson/src/vision/tensorrt_native.py	TensorRT engine I/O 검증, CUDA buffer·stream, execute_async_v3, H2D/D2H 및 동기화
jetson/src/vision/tensorrt_preprocess.py	640 정사각 letterbox, padding 114, BGR→RGB, CHW batch1, /255
jetson/src/vision/tensorrt_postprocess.py	출력 shape/dtype 검증, confidence, class-aware NMS, 원본 좌표 복원
jetson/src/vision/engine_inspector.py	M2 → pose → rectify → M1 → slot 판정과 stage timing 기록
jetson/INSTALL_OPERATING_KO.md	현재 운영본을 다른 Orin Nano 에 설치·검증하는 공식 절차
jetson/products/*/manifest.json	모델 파일, SHA, 입력/출력, 전처리·후처리 계약

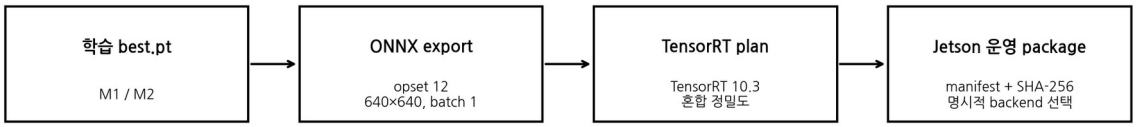
설계 원칙 운영 TensorRT 경로가 실패했다고 해서 PyTorch 로 조용히 대체하지 않는다. 평가자는 실제 어떤 backend 가 실행됐는지 runtime metadata 와 manifest 로 확인할 수 있다.

서비스가 시작될 때 검증되는 것

- standalone_service.py 는 package·station·line_profile 경로가 장치 로컬 범위인지 확인한다.
- PLC IP 와 local IP, profile mode, 최소 저장공간, bounded deadline 등 현장 계약을 확인한다.
- --check-only 는 실제 서비스 운전 전 파일·package·설정·모델 의존성 확인에 사용한다.

03 모델 계보: 학습 PT 에서 운영 TensorRT 까지

모델 변환·배포 계보



파일명 추정이 아니라 export/build/product manifest와 SHA-256 체인으로 연결

단계	M1 부품 검출	M2 엔진 전체 검출
PT SHA-256	42db183006ad2a4f61dc9c22e7a61a5b506cdacb5099004fc7c3b73b74bf5259	cf117defe782e1ae8b355e0ed13c014fc6f1538aa2dcdbe0fb6cfa54200f7755
ONNX SHA-256	433b16fd1e802bdf2cb0afba6bb6487a839d94b5adc286185b93fd3953b9c296	5cb024bb911f9357e46c12c3d2a7e4b5f19af20e5692013037c9235e0deca830
TRT plan SHA-256	743ce54b4d5c7b8a36addf3dbff50fa693ec4a509b31b476f68efc69f1bf2d6c	c82f2658ac20d06985a647c15e551c681e394451cfa1c53baed64f639bcebf83
운영 package	dynamic_parts_005	envelope_v007_fp16
입력 / 출력	[1,3,640,640] / [1,7,8400]	[1,3,640,640] / [1,5,8400]

중요 위 연결은 파일명 추정이 아니라 export 기록 → build_manifest → 제품 manifest → release/Jetson 관측의 SHA-256 대조에 기반한다.

GitHub 의 jetson/products/README.md 는 현재 두 TensorRT plan 을 실제 바이너리로 포함하며, 다른 GPU/TensorRT 환경으로 옮길 경우 원본 모델에서 재빌드·재검증해야 한다고 명시한다.

04 기록된 ONNX·TensorRT 변환 조건

구분	기록된 조건
PT → ONNX	Ultralytics YOLO.export 8.4.146, imgsz=640, batch=1, dynamic=False, half=False, nms=False, simplify=False, opset=12
ONNX → TensorRT	TensorRT 10.3 Builder API, workspace 1 GiB, optimization_level=3, max_aux_streams=0, TF32=False, precision_constraints=OBEY
M1 정밀도	FP16 + head FP32. build 기록에서 FP32 지정 레이어 77 개
M2 정밀도	FP16 + backbone FP32. build 기록에서 FP32 지정 레이어 208 개
I/O	TensorRT engine 입력·출력은 float32 로 검증
운영 파일	새 엔진을 임의로 재빌드하지 않고 현재 제품 package 의 .plan 을 사용

변환 절차를 평가자가 보는 순서

- 1) 학습 best.pt 의 SHA 를 확인한다.
- 2) export 기록의 source PT SHA 와 생성 ONNX SHA 를 확인한다.
- 3) build_manifest 에서 ONNX 입력과 생성 plan SHA, TensorRT 설정을 확인한다.
- 4) 제품 manifest 의 model artifact SHA 와 plan 을 대조한다.
- 5) release.json 과 실제 Jetson 관측 해시로 현재 운영 파일까지 연결한다.

표현 주의 두 plan 은 “순수 FP16”이라고 단순화하면 안 된다. 혼합 정밀도 정책이며, 파일 크기 자체도 PT 보다 작다고 볼 수 없다. 이번 최적화의 뚜렷한 장점은 파일 용량이 아니라 실행 프로세스 RSS 감소였다.

05 입력 전처리 계약

TensorRT 경로는 입력의 의미가 달라지지 않도록 정적 640 letterbox 계약을 코드로 고정한다. tensorrt_preprocess.py 의 계약을 어기면 INPUT_CONTRACT_MISMATCH 로 실패한다.

항목	운영 계약
원본	uint8 HWC BGR, 3 채널
resize	gain=min(640/H, 640/W), 선형 보간
letterbox	640×640 중앙 배치, padding value 114
채널/형태	BGR→RGB, HWC→CHW, batch dimension 추가
정규화	float32 또는 선언 dtype 으로 변환 후 /255
메모리	contiguous array
형상 기록	original_shape, resized_shape, gain, left/top/right/bottom padding 을 ImageTransform 에 보존

```
# tensorrt_preprocess.py 핵심 계약
gain = min(640 / height, 640 / width)
padded = cv2.copyMakeBorder(..., value=(114,114,114))
tensor = np.ascontiguousarray(padded[:, :, ::-1].transpose(2,0,1)[None], dtype=dtype)
tensor /= 255
```

왜 필요한가 PT 와 TensorRT 를 비교할 때 같은 imgsz=640 이라는 사실만으로 같은 입력이 되는 것은 아니다. resize·padding·채널 순서·정규화까지 같은 계약이어야 출력 차이를 변환 효과로 해석할 수 있다.

06 출력 후처리와 좌표 복원

tensorrt_postprocess.py 는 raw head 의 출력 이름·shape·dtype·finite 여부를 검사한 뒤 confidence → class-aware NMS → 원본 좌표 복원 순서로 처리한다.

단계	검증·처리 내용
출력 계약	manifest 의 tensor 이름·shape·dtype 과 실제 반환값이 정확히 일치해야 함
confidence	각 후보에서 argmax class 와 best score 를 선택하고 선언 threshold 보다 큰 후보만 사용
NMS	같은 클래스끼리 IoU 기준으로 중복 제거, max_det 300
좌표 복원	letterbox padding 을 빼고 gain 으로 나눈 뒤 원본 W×H 범위로 clip
비정상 좌표	복원 후 면적이 0 이거나 비정상 값이면 DetectorError
진단 메타데이터	raw candidate, eligible, kept, suppressed, padding/gain, confidence/NMS threshold 저장 가능

현재 운영 NMS IoU 는 0.45 이다. Benchmark 는 PT 와 TensorRT 에 같은 후처리를 적용해 E03 72 장 전체를 비교했다.

검출과 제품 판정의 구분 M1/M2 의 box 검출은 최종 제품 판정의 입력이다. 제품 판정에는 M2 pose, rectification, M1 부품 검출, 자리 매핑, 3 프레임 집계가 추가된다.

07 Native TensorRT 실행 경로

NativeTensorRTExecutor 는 engine 의 I/O 를 로드할 때 이름·개수·shape·dtype·TensorIOMode·DEVICE 위치·LINEAR format 을 검증한다. 동적/잘못된 차원 또는 예상하지 않은 I/O 는 실행 전에 거부한다.

순서	실행 내용
1. Host 준비	contiguous 입력을 pinned host buffer 에 복사
2. H2D	CUDA non-default stream 에서 device input 으로 비동기 복사
3. enqueue	context.execute_async_v3(stream_handle=...) 실행
4. synchronize	GPU 작업 완료를 동기화하고 완료 시점을 기록
5. D2H	output device → host 비동기 복사 후 다시 동기화
6. 후처리	NumPy output 을 복사해 TensorRTDetector/postprocess 로 전달
7. 메타데이터	engine SHA, TensorRT version, I/O, execute_count, last_timing 을 JSON 가능 구조로 제공

```
execute_async_v3(stream_handle=self.stream)
_synchronize()
copy_async(output_host, output_device, ..., D2H)
_synchronize()
return {output_name: output_array.copy()}
```

코드의 last_timing 은 host 관측 구간이며 순수 GPU kernel time 이라고 명시하지 않는다. Benchmark 도 별도 GPU kernel 단독시간을 측정하지 않았다.

08 M2 → Pose → M1 → 3 프레임 판정

engine_inspector.py 는 두 검출기를 독립적으로 호출하는 수준을 넘어서, 엔진 전체 검출을 부품 검출의 좌표계 정렬에 사용한다.

처리 단계	실제 코드 동작
M2 envelope	현재 frame 에서 product_envelope 검출
pose	유효한 product 가 1 개일 때 EnginePoseEstimator 로 자세 추정
rectify	pose 가 reliable 이면 canonical image 로 변환
M1 parts	canonical frame 에서 필수 부품 검출
restore	부품 검출 좌표를 pose matrix 로 원본 camera 좌표에 복원
slot decision	엔진 전체·부품 detections 를 inspect_frame 에 전달하여 자리 상태 판단
3-frame aggregate	세 프레임의 자리별 상태가 일치하는지 확인해 PASS/FAIL/REVIEW/ERROR 확정

운영 의미 최종 PASS/FAIL 은 “YOLO 한 번의 결과”가 아니다. 엔진 영역의 자세를 잡고, 정렬된 입력에서 필수 부품을 찾은 다음, 연속 세 프레임의 일관성을 확인하는 검사 파이프라인이다.

engine_inspector 는 product_detection_ms, pose_ms, rectification_ms, part_detection_ms, slot_decision_ms 와 engine_processing_ms 를 단계별 진단값으로 남길 수 있다.

09 현재 운영본 설치·기동 절차

다른 Jetson 에는 개발 소스를 재조합하지 않고 portable/operating_runtime.tar.gz 와 고정 release 를 기준으로 설치하도록 문서화돼 있다. 기존 setup.sh/install.py 의 CORE MOCK 경로와 구분한다.

단계	대표 명령 / 확인
1. 설치 입력 검증	python3.10 -B -X utf8 jetson/install_operating.py --verify
2. 새 장치 폴더 준비	install_operating.py --prepare --camera <by-id> --station-id <id>
3. Python 환경	python3.10 -m venv --system-site-packages ~/oned_device_bench/envs/app_v1
4. 운영 점검	check_operating.py / standalone_service.py --check-only / check_operating.py --gpu-smoke
5. 기준 등록	launch_live.py 로 STOP 상태에서 기준 엔진과 검사 영역 등록
6. PLC 활성화 전 확인	--enable-plc --confirm-stopped --confirm-camera-setup --confirm-plc-contract
7. systemd	engine-inspector.service 등록 후 enable --now, status/journal 확인
8. 현장 수용	정상·알려진 결품 실물로 원본 3 장·판정·DB·Result/Done·실제 배출 확인

설치와 현장 검증의 구분 파일 복사·GPU smoke 가 성공해도 새 장치의 카메라 구도·PLC·정상/결품 현장 수용시험까지 완료됐다는 뜻은 아니다.

10 최적화 결과와 운영 적용 확인

동일 Jetson·동일 E03 72 장·동일 전후처리에서 PT FP32 와 운영 TensorRT 를 직접 비교한 Benchmark 가 별도 증빙으로 존재한다.

항목	M1 부품 검출	M2 엔진 전체 검출
평균 지연시간	29.313 → 29.022 ms	29.480 → 30.266 ms
처리량	34.06 → 34.41 image/s	33.87 → 33.00 image/s
프로세스 RSS 관측 최대	1,250.5 → 636.9 MiB (약 49.1% 감소)	1,263.8 → 649.7 MiB (약 48.6% 감소)
고정 조건 TP/FP/FN	PT·TRT 모두 192/0/0	PT·TRT 모두 72/0/0
mAP50-95	0.639911 → 0.640873	0.836717 → 0.836629

속도는 M1 약 1% 감소, M2 약 2.7% 증가로 큰 가속을 확인하지 못했다. 반면 프로세스 RSS 관측 최대는 두 모델 모두 약 절반 수준으로 낮아졌다. 이 수치는 GPU 전용 메모리 절감이 아니라 라이브러리를 포함한 프로세스 RSS 관측값이다.

평가 포인트 변환 수행 자체보다 중요한 것은 같은 모델 계통을 동일 조건에서 다시 측정하고, 속도·메모리·정확도 변화를 함께 해석했다는 점이다.

11 구현 강점과 적용 범위

구분	평가자가 확인할 수 있는 강점
모델 계보	학습 checkpoint 부터 ONNX, TensorRT, 제품 package, 장치 release 까지 SHA-256 연결
명시적 backend	manifest 계약에 따라 adapter 를 선택하고 silent fallback 금지
고정 입출력 계약	전처리·출력 shape/dtype·NMS·좌표 복원 조건을 코드로 검사
단일 운영 supervisor	systemd 서비스 하나가 카메라·추론·PLC child·웹을 감독
복합 검사 파이프라인	M2 자세 정렬 후 M1 부품 검출, 3 프레임 일관성으로 최종 판정
재현·운영 분리	portable 설치, check-only, GPU smoke, 현장 기준 등록, PLC 활성화를 단계적으로 분리
정량 검증	동일 Jetson Benchmark 와 실물 7 건 통합시험·DB 조회 증빙을 별도 보존

현재 범위를 넘어서는 주장

- TensorRT 가 모든 조건에서 PT 보다 빠르다는 주장은 하지 않는다.
- Benchmark 의 72 장은 E03 개발 검증 자료이며 독립 final test 가 아니다.
- 다른 Jetson 에 portable 설치만 했다고 현장 수용시험까지 끝난 것으로 보지 않는다.
- 현재 저장소의 TensorRT plan 은 생성 환경 의존성이 있으므로 다른 GPU/TensorRT 환경에서는 재빌드·재검증이 필요하다.

부록 A 평가 근거 찾아보기

ID	근거	확인 내용
G1	GitHub master cfaa372eece0cd6a7dc659cc316c256fb4b6af76	현재 제출 기준 소스 스냅샷
G2	jetson/INSTALL_OPERATING_KO.md	portable 설치, 환경, GPU smoke, 기준 등록, PLC/service 절차
G3	jetson/INTEGRATED_RUNTIME.md	단일 supervisor, OpENer, runtime·journal 운영 구조
G4	jetson/standalone_service.py	서비스 진입점, 설정/PLC 계약 검증, runtime·웹 구성
G5	jetson/src/vision/detector_factory.py	TensorRT/PyTorch 명시 선택과 fallback 금지
G6	jetson/src/vision/ tensorrt_{native,preprocess,postprocess}.py	native TRT 실행·입력/출력 계약·NMS·좌표 복원
G7	jetson/src/vision/engine_inspector.py	M2→pose→rectify→M1→slot 판정
G8	jetson/products/README.md 및 각 manifest.json	현재 운영 TensorRT plan·환경 의존성·모델 출처
B1	04_엔진모형_최적화전후_Benchmark_보고서 v1.1 + 원본 증빙	변환 설정·해시·Jetson 동일 조건 Benchmark
M1	03_엔진모형_모델_평가서 v1.1	학습·개발검증 및 PT→운영 모델 연결
I1	05_엔진모형_통합테스트_결과서 v1.0	실물 7 건에서 판정·PLC·DB 연결
D1	06_엔진모형_DB_구조와_조회방식 v1.0	검사 이력 저장·조회 구조

GitHub 에서 바로 확인할 경로

- jetson/INSTALL_OPERATING_KO.md
- jetson/INTEGRATED_RUNTIME.md
- jetson/standalone_service.py
- jetson/src/vision/detector_factory.py
- jetson/src/vision/tensorrt_native.py
- jetson/src/vision/tensorrt_preprocess.py
- jetson/src/vision/tensorrt_postprocess.py
- jetson/src/vision/engine_inspector.py
- jetson/products/ENGINE_Z3005_5/dynamic_parts_005/manifest.json
- jetson/products/ENGINE_Z3005_5/envelope_v007_fp16/manifest.json

문서 작성 기준: GitHub master 를 2026-10-20 에 다시 읽어 현재 commit 을 확인하고, 앞서 보존한 모델 평가·Benchmark·통합시험 증빙과 대조했다. 이 문서 작성 과정에서 Jetson 재배포·서비스 재시작·PLC 운전·새 추론은 수행하지 않았다.